

Capa 1: Template + Merge por Cliente

Estrategia de sincronización para proyectos frontend — Agencia SaaS
Documento técnico · Nivel 1 de 3

Concepto clave: Separar *estructura de presentación*. La estructura (componentes, fetching, layouts) se comparte. El CSS y diseño de cada cliente **siempre** queda en su repositorio y nunca se sobrescribe automáticamente.

1. Visión General

Cada cliente nuevo se crea a partir de un **template repo** (`astro-starter` o `nextjs-starter`). El template queda configurado como un **git remote** adicional en cada proyecto cliente. Cuando el template evoluciona, se hace `git merge` desde ese remote para traer los cambios nuevos.

Los conflictos que aparecen en el merge son esperables y deseables: representan las personalizaciones de cada cliente. El merge force a revisar caso por caso si el cliente adopta el cambio nuevo o mantiene su versión.

2. Setup Inicial del Template

Creás el repo del template **una sola vez** por framework:

```
mkdir astro-starter && cd astro-starter
git init
# Agregás toda la estructura base
git add . && git commit -m "feat: initial astro starter"
```

```
git remote add origin git@github.com:tu/astro-starter.git
git push -u origin main
```

El template incluye:

- **Layout base** — estructura HTML, SEO, meta tags
- **Componentes compartidos** — HeroBlock, TextBlock, FooterBlock, etc.
- **Cliente Payload API** — `src/lib/payload.ts`
- **Config** — TypeScript, ESLint, CI/CD
- **CSS base** — reset, variables de ejemplo, tipografía

3. Crear un Cliente Nuevo

```
# 1 - Clonás el template como base
git clone git@github.com:tu/astro-starter.git restaurantepepito.com
cd restaurantepepito.com

# 2 - Renombrás el remote para que el template quede como remote adicional
git remote rename origin template
git remote add origin git@github.com:tu/restaurantepepito.com.git

# 3 - Primer push al remote del cliente
git push -u origin main

# 4 - A partir de acá trabajás normal:
#   - Cambiás CSS, colores, contenido
#   - Configurás dominio y tenant slug
#   - Hacés commits en origin (tu remote)
```

El resultado es:

```
$ git remote -v
origin    git@github.com:tu/restaurantepepito.com.git  (fetch)
origin    git@github.com:tu/restaurantepepito.com.git  (push)
template  git@github.com:tu/astro-starter.git          (fetch)
↑ solo lectura - nunca pusheás al template desde un cliente
```

4. Actualizar Todos los Clientes

Cuando el template cambia (nuevo componente, mejora en el cliente Payload, bugfix):

⚠ **Esto se hace por cada cliente, uno a uno.** Para 5-10 clientes es viable. Más allá, necesitás la Capa 2.

```
cd restaurantepepito.com

# 1 – Traer los cambios del template
git fetch template

# 2 – Mergear los cambios a tu rama principal
git merge template/main

# 3 – Resolver conflictos (ver sección 5)
# 4 – Commit del merge
git commit

# 5 – Pushear al cliente
git push origin main
```

5. Conflictos: Qué Esperar

No todos los archivos generan conflicto. La clave está en cómo estructure el template desde el inicio:

Archivo	Conflicto	Qué pasa
<code>src/styles/variables.css</code>	✓ Siempre	El cliente cambió colores, fuentes, espaciado. Hay que mezclar los cambios del template (nuevas variables) con las del cliente.
<code>src/styles/theme.css</code>	✓ Siempre	CSS custom del cliente. Hay que decidir si incorporar los cambios del template.
<code>src/components/*/styles.css</code>	✓ Probable	Cada cliente tiene su diseño por componente. Template puede traer mejoras estructurales.

Archivo	Conflicto	Qué pasa
<code>src/components/*/index.astro</code>	✗ Raro	Si el cliente nunca tocó la lógica del componente, mergea limpio.
<code>src/lib/payload.ts</code>	✗ No	Nadie toca esto salvo vos. Mergea siempre limpio.
<code>package.json</code>	⚠ A veces	Template agregó/quité dependencias. Cliente puede tener las suyas. Se resuelve fácil.

Los conflictos no son un problema, son el feature. Te obligan a decidir conscientemente qué cambios del template aplicás a cada cliente. Un merge sin conflictos significaría que el cliente no tiene personalización real — y en este negocio, cada cliente es único.

6. Estructura Ideal del Template

Para minimizar conflictos sin perder control, separás claramente los archivos que el template controla de los que el cliente personaliza:

```
src/
├── components/
│   ├── HeroBlock/
│   │   ├── index.astro    ← Template (lógica, estructura HTML)
│   │   └── styles.css     ← Cliente (diseño visual)
│   └── FooterBlock/
│       ├── index.astro    ← Template
│       └── styles.css     ← Cliente
│   └── shared/
│       ├── Layout.astro   ← Template (nadie toca)
│       └── SeoHead.astro  ← Template (nadie toca)
├── lib/
│   └── payload.ts         ← Template (nadie toca)
├── styles/
│   ├── global.css        ← Template (resetea base)
│   ├── variables.css     ← CLIENTE (colores, fuentes)
│   └── theme.css          ← CLIENTE (tema custom)
└── content/
```

```
| └─ site.config.ts      ← CLIENTE (tenant slug, dominio)
| └─ astro.config.mjs   ← Template
```

⚠ **Regla fundamental:** Si un archivo es del cliente (CSS, config específica), ponelo en una ubicación obvia y documentalo. Nunca mezcles lógica de template con CSS custom en el mismo archivo.

7. Flujo de Trabajo Real

Día a día (sin cambios de template)

- El cliente trabaja en su repo normalmente
- Commits, push, deploy — sin intervención del template

Cuando el template cambia

1. Hacés el cambio en el repo del template
2. Lo pusheás a `template/main`
3. Agarrás la lista de clientes activos
4. Uno por uno: `git fetch template && git merge template/main`
5. Resolvés conflictos, commiteás, pusheás
6. Verificás que el build ande (`pnpm build`)
7. Pasás al siguiente cliente

Ejemplo concreto

Agregaste un `CookieConsentBanner` al template. Los pasos:

```
# 1 – Modificás el template
cd astro-starter
# Agregás: src/components/shared/CookieConsent.astro
# Agregás: src/components/shared/styles-cookie.css
git add . && git commit -m "feat: add cookie consent banner"
git push

# 2 – Por cada cliente:
cd ../restaurantepepito.com
git fetch template
git merge template/main
# Resolvés conflicto en styles-cookie.css si el cliente
```

```
#  modificó estilos globales que se solapan
# 0 no hay conflicto porque es un archivo nuevo
git commit
git push

cd ../tienda-moda
git fetch template
git merge template/main
...
```

8. Automatización Mínima (Opcional)

Si querés ahorrar tiempo sin llegar a la Capa 2, podés hacer un script bash que itere sobre los clientes:

```
#!/bin/bash
# update-all.sh
CLIENTS=(
  "restaurantepepito.com"
  "tienda-moda"
  "academia-online"
)

for client in "${CLIENTS[@]}; do
  echo "=== Updating $client ==="
  cd "$client"
  git fetch template
  if git merge template/main --no-commit --no-ff; then
    git commit -m "chore: merge template updates"
    git push
    echo "✓ $client – merged successfully"
  else
    echo "✗ $client – CONFLICTS. Resolve manually."
    git merge --abort
  fi
  cd ..
done
```

⚠ **Este script no resuelve conflictos.** Si un cliente tiene conflictos, aborta el merge y te avisa. Los conflictos los resolvés manualmente. Esto es deliberado — un merge automático que sobrescriba CSS de un cliente es un error de producción.

9. ¿Hasta Cuántos Clientes?

Clientes	Viable	Realidad
1-5	FÁCIL	Un merge por cliente te lleva 5-10 minutos. En total < 1 hora.
5-10	OK	Sigue siendo viable si los cambios de template no son masivos. Si cada update toca 20 archivos, empieza a doler.
10-20	LÍMITE	Un cambio semanal de template te consume medio día. Empezás a saltar clientes. Es hora de la Capa 2.
20+	ROTO	No lo vas a mantener. Necesitás paquetes npm compartidos YA.

10. Señales para Migrar a Capa 2

1. Un merge de template te lleva más de 30 minutos por cliente
2. Empezás a preguntarte "¿a este cliente ya le apliqué el cambio del CookieConsent?"
3. El `src/lib/payload.ts` cambia seguido (nuevos endpoints, cache, error handling)
4. Alguien que no sos vos necesita aplicar los merges
5. Hay clientes con versiones distintas del template y perdiste la cuenta

Cuando pase, movés `src/lib/payload.ts`, los tipos compartidos, y los componentes sin CSS custom a paquetes npm. El merge de template se simplifica drásticamente porque esos archivos ya no existen en el template ni en los clientes — viven en `node_modules` y se actualizan con `pnpm update`.

Apéndice A: Comandos Rápidos

Acción	Comando
Crear cliente	<code>git clone template && git remote rename origin template && git remote add origin <nuevo></code>
Ver remotes	<code>git remote -v</code>
Traer cambios del template	<code>git fetch template</code>
Ver qué cambió en el template	<code>git log template/main --oneline -10</code>
Ver diff contra template sin mergear	<code>git diff HEAD..template/main</code>
Mergear template	<code>git merge template/main</code>
Abortar merge con conflictos	<code>git merge --abort</code>
Listar archivos en conflicto	<code>git diff --name-only --diff-filter=U</code>